开源软件基础

基于 Flask 的 Web 开发

Zhilei Ren

OSCAR Team, School of Software, Dalian University of Technology

December 18, 2017

Outline

安装

Flask

- pip install flask
- conda install flask

A Minimal Application

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def hello_world():
    return 'Hello, World!'
```

A Minimal Application

启动服务器

- 保存文件为 hello.py
- export FLASK_APP=hello.py (Linux/OSX)
- set FLASK_APP=hello.py (Windows)
- flask run
- 使用--host=0.0.0.0 使能跨 IP 访问

A Minimal Application

```
$ export FLASK_APP=hello.py (run set FLASK_APP=hello.p
$ flask run
 * Running on http://127.0.0.1:5000/
```

A Minimal Application

This launches a very simple builtin server, which is good enough for testing but probably not what you want to use in production. For deployment options see Deployment Options.

A Minimal Application

```
flask run --host=0.0.0.0
```

Outline

Routing

路由

- 使用 @app.route() 标注 URL 中的链接
- 函数的返回值响应 GET/POST 请求
- 建立 URL 与函数间的绑定
- 使用 <variable> 与函数参数表达动态 URL

Routing

```
@app.route('/')
def index():
    return 'Index Page'

@app.route('/hello')
def hello():
    return 'Hello, World'
```

Routing

```
@app.route('/user/<username>')
def show_user_profile(username):
    # show the user profile for that user
    return 'User %s' % username

@app.route('/post/<int:post_id>')
def show_post(post_id):
    # show the post with the given id, the id is an in
    return 'Post %d' % post_id
```

Routing

HTTP (the protocol web applications are speaking) knows different methods for accessing URLs. By default, a route only answers to GET requests, but that can be changed by providing the methods argument to the route() decorator. Here are some examples:

Routing

```
from flask import request

@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        do_the_login()
    else:
        show_the_login_form()
```

Outline

Rendering Templates

Generating HTML from within Python is not fun, and actually pretty cumbersome because you have to do the HTML escaping on your own to keep the application secure. Because of that Flask configures the Jinja2 template engine for you automatically.

Rendering Templates

To render a template you can use the render_template() method. All you have to do is provide the name of the template and the variables you want to pass to the template engine as keyword arguments. Heres a simple example of how to render a template:

Rendering Templates

```
from flask import render_template

@app.route('/hello/')
@app.route('/hello/<name>')
def hello(name=None):
    return render_template('hello.html', name=name)
```

Rendering Templates

```
/application.py
/templates
    /hello.html
```

Rendering Templates

Case 2: a package:

Rendering Templates

```
/application
    /__init__.py
    /templates
        /hello.html
```

Rendering Templates

```
<!doctype html>
<title>Hello from Flask</title>
{% if name %}
  <h1>Hello {{ name }}!</h1>
{% else %}
  <h1>Hello, World!</h1>
{% endif %}
```

Outline

Accessing Request Data

```
@app.route('/login', methods=['POST', 'GET'])
def login():
    error = None
    if request.method == 'POST':
        if valid_login(request.form['username'],
                       request.form['password']):
            return log_the_user_in(request.form['usern
        else:
            error = 'Invalid username/password'
    # the code below is executed if the request method
    # was GET or the credentials were invalid
    return render_template('login.html', error=error)
```

Accessing Request Data

```
searchword = request.args.get('key', '')
```

Outline

Redirects and Errors

To redirect a user to another endpoint, use the redirect() function; to abort a request early with an error code, use the abort() function:

Redirects and Errors

```
from flask import abort, redirect, url_for

@app.route('/')
def index():
    return redirect(url_for('login'))

@app.route('/login')
def login():
    abort(401)
    this_is_never_executed()
```

Redirects and Errors

This is a rather pointless example because a user will be redirected from the index to a page they cannot access (401 means access denied) but it shows how that works.

Outline

Sessions

In addition to the request object there is also a second object called session which allows you to store information specific to a user from one request to the next. This is implemented on top of cookies for you and signs the cookies cryptographically. What this means is that the user could look at the contents of your cookie but not modify it, unless they know the secret key used for signing.

Sessions

```
from flask import Flask, session, redirect, url_for, e

app = Flask(__name__)

@app.route('/')
def index():
    if 'username' in session:
        return 'Logged in as %s' % escape(session['use
    return 'You are not logged in'

@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        session['username'] = request.form['username']
```

Sessions

```
>>> import os
>>> os.urandom(24)
'\xfd{H\xe5<\x95\xf9\xe3\x96.5\xd1\x01O<!\xd5\xa2\xa0\

Just take that thing and copy/paste it into your code
```